2025 IEEE International Conference on Cloud Engineering (IC2E) | 979-8-3315-3465-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/IC2E65552.2025.00047

Performance-Aware Scheduling and Load Balancing for Edge/Cloud Systems

Panagiotis Giannakopoulos*‡, Bart van Knippenberg†§, Kishor Chandra Joshi*‡, Nicola Calabretta*‡, George Exarchakos*‡
‡Electrical Engineering, Eindhoven University of Technology, Eindhoven, The Netherlands
§Materials and Structural Analysis, Thermo Fisher Scientific, Eindhoven, The Netherlands
Emails: *{p.giannakopoulos,k.c.joshi,n.calabretta,g.exarchakos}@tue.nl, †bart.vanknippenberg@thermofisher.com

***Abstract*—Edge computing enables time-sensitive applications by bringing computation closer to data sources. However, existing orchestration frameworks such as Kubernetes often overlook performance variability, leading to inefficient resource management. While recent advances have introduced performance-aware scheduling, load balancing remains largely reactive. In particular, performance-aware load balancers are still missing from mainstream systems, limiting their ability to meet subsecond latency requirements. This paper presents a unified extension to orchestration frameworks that integrates both performance-aware scheduling and load balancing. We design lightweight performance predictors, trained on historical monitoring data, to estimate performance fluctuations. Variability predictors inform the scheduler, while Round-Trip Time (RTT) predictors guide load balancing decisions. Our approach is validated using a Single Particle Analysis (SPA) application deployed on a Kubernetes-based edge computing testbed. Results demonstrate that the predictors achieve approximately 85% accuracy in capturing RTT variability. Simulation results show reductions in application completion times ranging from 50% to 78% for scheduling and 68% for load balancing, highlighting the effectiveness of predictive orchestration in dynamic edge/cloud environments.**

***Index Terms*—Edge computing, Performance prediction, Performance-aware scheduling, Performance-aware load-balancing, edge-cloud continuum, Kubernetes, Prometheus**

I. Introduction

The increasing convergence of High-Performance Computing (HPC), cloud, and edge infrastructures is transforming the landscape of distributed computing. This unified computing continuum enables a wide range of applications, including time-sensitive services such as real-time analytics, autonomous systems, and interactive AI workloads at the network edge. These applications operate under strict time and resource constraints, making consistent and predictable performance a critical requirement [1], [2].

Orchestrating workloads across such heterogeneous environments presents significant challenges. Resources range from high-performance cloud servers to resource-constrained edge nodes, all of which are prone to temporal performance fluctuations due to co-location interference [3], architectural differences [4], and dynamic background load [1]. Traditional orchestration systems, such as Kubernetes, typically rely on static placement policies or reactive metrics (e.g., CPU or memory utilization), which are insufficient for capturing short-term performance shifts. These coarse-grained strategies often result in suboptimal placements, increased tail latencies, and degraded Quality of Service (QoS) for time-sensitive applications [2].

We address the challenges of unpredictable performance fluctuations caused by heterogeneous nodes, co-location interference, and the limitations of static or reactive orchestration policies. To this end, we propose a performance-aware orchestration framework for hybrid edge/cloud infrastructures. Our architecture augments conventional orchestrators with predictive capabilities that proactively mitigate performance variability. We develop lightweight predictors trained on historical monitoring data (e.g., CPU usage, memory, and network activity) and Round-Trip Time (RTT) measurements to forecast application behavior. These predictions enable scheduling and load balancing decisions that are both performance-aware and adaptive to dynamic system conditions.

By anticipating fluctuations in performance, the orchestrator dynamically adapts to runtime conditions, improving responsiveness and reducing task completion times. We validate our framework using a scientific workflow from electron microscopy (EM), specifically the Single Particle Analysis (SPA) pipeline. SPA workloads are well-suited for this study as they exhibit diverse and bursty resource demands and execution times ranging from milliseconds to several seconds, posing a realistic challenge for performance-aware orchestration.

The main contributions of this work are as follows. We design a predictive orchestration framework that integrates lightweight RTT and variability predictors into existing mechanisms, enabling joint performance-aware scheduling and load balancing in heterogeneous edge/cloud environments. We show that these predictors achieve about 85% accuracy with minimal inference overhead on a real Kubernetes-based edge testbed. Simulations further demonstrate that integrating them into orchestration policies reduces application completion times by up to 78% for scheduling and 68% for load balancing, highlighting their effectiveness in mitigating performance variability under dynamic workloads.

The remainder of this paper is structured as follows: Section II reviews related work. Section III outlines key challenges. Section IV introduces the use case. Section V provides the background. Section VI presents our architecture. Section VII details the setup. Section VIII discusses the results, and Section IX concludes with future directions.

2694-0825/25/$31.00 ©2025 IEEE
DOI 10.1109/IC2E65552.2025.00047

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

II. Related Work

Improving the decision-making mechanisms in edge and cloud systems is essential to enhance application performance. These mechanisms include the scheduler (maps applications to servers), the load balancer (routes incoming requests to specific replicas), and the autoscaler (adjusts the number of active replicas). We review the existing work on these components, along with approaches to performance prediction that support informed decision-making.

A. Performance Prediction

Performance prediction in distributed systems has been widely studied as a way to inform resource management decisions. Mohamed et al. [5] proposed ML-based models using monitoring metrics to predict microservice latency in Kubernetes. Fu et al. [6] estimated remaining execution time of containerized applications to inform progress-aware scheduling. Other works focus on CPU benchmarking [7], volunteer computing [8], or network behavior [9]. However, most existing approaches either ignore co-location effects—critical in shared edge environments—or use a fixed, often arbitrary set of monitoring metrics, limiting adaptability and accuracy.

In contrast, our approach trains lightweight, per-node and per-application predictors that are informed by co-location scenarios and driven by dynamically selected metrics using a specialized correlation tool. These predictors are explicitly designed to run in runtime with microsecond-level inference latency, making them suitable for time-sensitive application deployed across heterogeneous edge-cloud clusters.

B. Scheduling

Several works have proposed ML-enhanced schedulers that improve application placement decisions. Quasar [10] uses machine learning to model applications and schedule them to satisfy QoS while optimizing resource use. Dauwe et al. [11] simulate co-location-aware scheduling that incorporates interference and energy modeling. Townend et al. [12] propose energy-efficient hardware-aware scheduling, while Harichane et al. [13] use classification to decide CPU vs GPU placement based on application profiles.

Our work builds upon these efforts by developing node-local predictors that explicitly estimate the impact of co-location on RTT variability for each application. By operating at the node level, these predictors account for configuration differences (i.e. CPU models) across nodes. Integrating these predictors into the scheduling pipeline enables sub-second, performance-aware placement of applications in dynamic edge–cloud environments.

C. Load Balancing

Conventional load balancing approaches in distributed systems, such as round-robin, random, or least-connections, are often stateless and reactive. Alankar et al. [14] showed that round-robin improved throughput in web server scenarios, while Laukka et al. [15] found least-connections outperformed other heuristics in mobile edge simulations. HLB [16] improved load distribution by considering real-time resource utilization and processing speeds. Chen and Yang [17] designed a hybrid context-aware scheduler for mobile edge computing that uses metadata and predictive models to preserve data locality.

While some of these works move toward adaptive strategies, they generally do not incorporate lightweight predictors of RTT for real-time routing. Our system introduces RTT predictors that estimate per-request latency based on live node-level metrics. These can be used to dynamically compute routing weights for each application replica, enabling proactive, decentralized load balancing that adapts to both hardware heterogeneity and application interference.

D. Autoscaling

Predicting workload trends is key to enabling proactive autoscaling, allowing applications to scale resources before overload or under-utilization occurs. Gao et al.[18] surveyed time-series forecasting techniques for cloud workloads, including auto-regressive models, Support Vector Machines (SVMs), and Long short-term memory (LSTM) networks. Xu et al.[19] and Yazdanian et al.[20] proposed deep learning ensembles for multivariate workload prediction, which can reduce over-provisioning and improve QoS by anticipating demand changes. These prediction methods are integrated into autoscaling frameworks to decide when and how much to scale. Toka et al.[21] further demonstrated adaptive autoscaling by dynamically selecting the best forecasting model per workload.

While autoscaling is not the primary focus of our study, our architecture incorporates workload predictors to estimate request arrival rates for each application. These forecasts can be used to support future integration of predictive autoscaling mechanisms, enabling unified orchestration that adapts replica count, placement, and routing jointly based on performance forecasts.

III. Challenges in Performance Predictability

Edge computing introduces significant performance variability due to five key challenges: system heterogeneity, application co-location, dynamic workloads, execution-time randomness, and resource limitations.

A. System Heterogeneity

Performance variability in edge and cloud environments arises from differences in hardware and software configurations, even when the same application is executed repeatedly on a single server [4]. Server-specific characteristics, such as CPU cores, memory, and capacity constraints, affect resource allocation, and thus impact application completion times. Moreover, performance may degrade over time as a result of aging of the hardware. Together, these factors complicate the prediction of performance. An approach is to develop machine learning models that incorporate hardware specifications (e.g., CPU model) to estimate execution times [7]. However, such models require large and diverse datasets for generalization.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

B. Application Co-location and Resource Contention

Edge and cloud servers frequently host multiple applications concurrently, leading to performance fluctuations caused by resource contention among co-located applications [3]. As applications are dynamically deployed or removed, even identical servers may exhibit varying behaviors. Additionally, since the nodes share network resources, applications running on different nodes can interfere with each other due to network contention. To address this, interference models can be developed by analyzing the impact of different co-location scenarios on performance [10]. Alternatively, performance can be estimated using monitoring metrics that capture the resource usage patterns of co-located applications [5].

C. Dynamic Workloads

Edge and cloud applications are typically event-driven, remaining idle until activated by user requests. The rate of incoming requests is highly dependent on user behavior, resulting in bursty and time-varying workloads that place uneven demands on shared resources. High request rates can lead to resource contention and poor application performance, while low request rates can cause resources to be under-utilized. To address these challenges, workload forecasting using models such as autoregressive techniques, SVM, or LSTM models [18] enables proactive resource scaling, helping maintain performance while optimizing resource utilization.

D. Application Execution Variability

The execution time of an application can exhibit variability even when running in isolation on the same server [4]. This is often due to inherent non-determinism in the internal algorithms of the application. Such variability poses challenges for accurate performance modeling, especially when internal behaviors are not exposed through monitoring metrics. However, this source of variability is generally smaller compared to external factors such as resource contention or hardware differences. To identify and isolate these intrinsic causes, particularly in microservice-based applications affected by code changes or failures, techniques such as microbenchmarking [22] and anomaly detection [23] can be used.

E. Resource Constraints

Nodes in edge environments often have limited processing, memory, and storage resources, which constrains both the complexity of performance predictors and the level of monitoring granularity that can be supported. While fine-grained metrics can improve prediction accuracy, they also introduce additional overhead. To maintain efficiency, lightweight models with a minimal yet informative feature set reduce monitoring and processing costs while preserving predictive performance.

IV. Usecase

In this study, we focus on the Single Particle Analysis (SPA) workflow, a common usecase in electron microscopy. A high-level overview of the SPA infrastructure is illustrated in Figure 1. SPA involves acquiring thousands of 2D image stacks

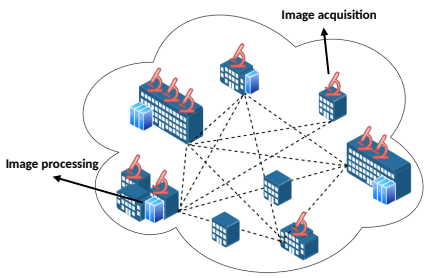

Fig. 1. High architecture of SPA usecase

from a vitrified sample, which can be dynamically optimized by processing the data with advanced algorithms and adjusting parameters for future acquisitions. For these adjustments to be effective, the algorithm results must be available within sub-seconds, requiring dedicated compute resources co-located with the electron microscope. However, the compute resources attached to a particular microscope often remain idle when not in use. By adopting a distributed platform, these resources can be pooled and dynamically reassigned among microscopes, allowing workloads from one instrument to be executed on idle nodes attached to another. This approach increases overall utilization for time-sensitive tasks. In addition, privacy and data security are critical in such usecases, making on-premise edge computing a compelling solution. Efficient scheduling of time-sensitive compute tasks must account for resource availability. Accurate prediction of task execution performance on available resources is crucial to enhance computational efficiency in these facilities.

V. Background

In this section, we provide an overview of the orchestration, observability, and traffic control mechanisms in edge and cloud systems, along with the tools integrated into our proposed architecture.

A. Orchestration and Scheduling

Container orchestration platforms automate the deployment and management of containerized applications by abstracting the infrastructure and implementing operational policies. They include components for scheduling, which decide where to place application instances (pods) based on resource requests. One widely used platform, Kubernetes [24], assigns workloads based on static resource metrics such as CPU and memory. However, it does not account for performance degradation due to resource contention. To address this, Kubernetes supports scheduler extensions, allowing integration of external performance models to improve placement decisions in dynamic environments.

B. Monitoring and Tracing

Observability is critical for understanding and managing the behavior of distributed systems. It involves collecting

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

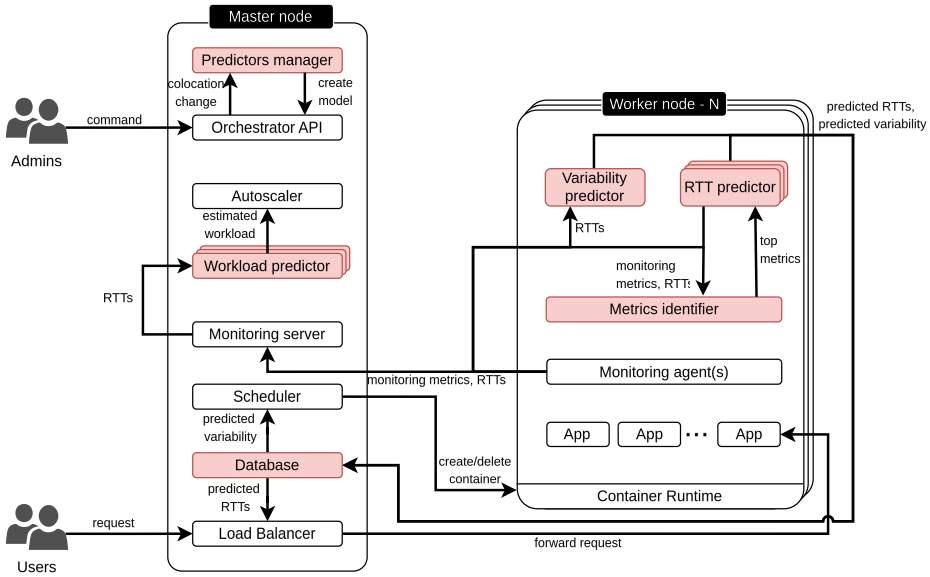

Fig. 2. High level architecture of the proposed performance-aware Kubernetes extension. The new components are colored as red.

time-series data and request-level traces to detect anomalies, diagnose bottlenecks, and support data-driven decisions. Monitoring systems gather system and application metrics over time. A widely adopted tool is Prometheus [25], which scrapes metrics from monitoring agents and stores them for querying and analysis. Monitoring agents expose node- and application-level metrics that feed into these systems. Tracing complements monitoring by capturing the path of individual requests as they traverse multiple services. Tools such as Jaeger [26] record spans and visualize request flows, allowing detailed latency analysis and root cause identification.

C. Traffic Management

Scalable architectures require mechanisms to distribute traffic across multiple application instances while maintaining responsiveness and availability. Load balancing strategies must consider runtime conditions to avoid overloading individual application instances. Service meshes implement advanced traffic control through lightweight proxies that intercept communication between services. Istio [27], for example, enables routing, retries, and failure recovery through configurable rules. By integrating external metrics, such as predicted response times, these meshes support dynamic and performance-aware routing decisions.

VI. Proposed Architecture

The architecture of the proposed performance-aware computing cluster, which incorporates performance predictors for both scheduling and load balancing, is illustrated in Figure 2. The cluster consists of physical and/or virtual machines acting as nodes, where applications are deployed within containers. For completeness, we also include an autoscaler component that interacts with workload predictors to maintain performance in the presence of fluctuating demand. The components introduced in our architecture are colored red. In the current stage of our work, the applications are stateless. In this context, a task refers to a request sent to an application that remains active until a response is delivered.

A. Metrics Identifier

Modern monitoring systems typically expose a large number of metrics, which complicates the development of performance predictors due to high dimensionality and computational burden. To address this, we use *perfCorrelate* [28], a tool designed to quantify the correlation between each monitoring metric and the performance of tasks. This tool enables the identification of a critical subset of metrics that best explain performance variability, facilitating the development of efficient and accurate predictive models. For each application m on node n, *perfCorrelate* identifies a subset of top metrics $\mathcal{X}_{n,m} = \{x_1, x_2, \ldots, x_k\}$ most relevant to its performance variability.

The *perfCorrelate* workflow involves three main steps. First, the time-series monitoring data (i.e. from Prometheus) are transformed into a feature space to allow their processing with conventional correlation techniques, such as the Pearson. Subsequently, redundant metrics are removed by analyzing inter-metric correlations, retaining only those that provide meaningful information for characterizing performance. In the final step, the correlation between each remaining metric and the task RTT is calculated, resulting in a refined set of highly relevant metrics. RTT values are obtained from the distributed tracing system, such as Jaeger. Each node runs a local instance of *perfCorrelate* to analyze the relationship between its own monitoring data and the performance of the target application.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

Monitoring data are collected from the local monitoring agents to eliminate network overhead. This approach allows us to select a concise yet relevant subset of monitoring metrics for training prediction models, rather than relying on an arbitrary or overly broad selection.

B. Performance Predictors

To optimize scheduling decisions, it is essential to employ accurate performance predictors. We treat applications as black boxes with no access to their internal code. Because each node may differ in hardware configuration, software environment, and co-located applications, performance characteristics vary significantly. To address this, we develop performance predictors that are both node-specific and application-specific, meaning that each unique combination of node and application is assigned its own dedicated predictor. This granularity allows our models to capture the unique behavior of each node and improve prediction accuracy. As the co-location of applications evolves, predictors are continuously updated with newly observed data, including RTT and corresponding monitoring metrics.

A *prediction manager* component is deployed to oversee application co-location and node changes. It creates new predictors when previously unseen nodes or applications are introduced. The prediction manager detects changes such as node additions or changes in co-located applications by periodically querying the orchestrator API. Our architecture defines two types of predictors: variability predictors, which estimate the expected mean and standard deviation of application performance, and RTT predictors, which predict the exact RTT for individual tasks. To ensure scalability and reduce communication overhead, performance predictors are hosted locally on each worker node. Their outputs are stored in a database and later accessed by the scheduling and load-balancing mechanisms. Each predictor retrieves the necessary data (i.e., RTT, monitoring metrics) from the local monitoring agent (i.e. Prometheus) of the corresponding node.

1) Variability Predictors: Variability predictors, denoted $V_{(n)}(\mathbf{A}_n)$, are defined per node n, where $\mathbf{A}_n$ is the set of applications currently active on that node. These predictors estimate the tuple $\big(\mu_{(n,m)}(\mathbf{A}_n), \sigma_{(n,m)}(\mathbf{A}_n)\big)$, representing the mean and standard deviation of the RTT for each application m under the given co-location.

The training dataset for these models comprises input vectors $X = (x_1, x_2, \ldots, x_M)$, where each element x_m denotes the number of replicas of application m present during a particular co-location scenario. The corresponding output vector is

$$Y = (\mu^1_{\text{RTT}}, \sigma^1_{\text{RTT}}, \mu^2_{\text{RTT}}, \sigma^2_{\text{RTT}}, \ldots, \mu^M_{\text{RTT}}, \sigma^M_{\text{RTT}}) \tag{1}$$

representing the average and standard deviation of the RTTs for each of the M applications.

Before training, the data are normalized to the range [0, 1], and outliers are removed using a z-score threshold of 3. The models considered include ridge regression [29], support vector machines (SVMs) [30], and random forests (RF) [31]. The final model is chosen by minimizing the Root Mean Square Error (RMSE) while ensuring that its inference time remains low (i.e. below one second). Each node deploys a single variability predictor to capture local performance fluctuations.

2) RTT Predictors: The RTT predictors are denoted as $y_{(n,m)} = f_{(n,m)}(X_{n,m})$, where $y_{(n,m)}$ denotes the predicted RTT of application m on node n. The input $X_{n,m}$ comprises the top monitoring metrics correlated with RTT, as identified by *perfCorrelate*. These metrics are extracted from a five-second monitoring window prior to task submission and transformed into statistical and temporal features. The function $f_{(n,m)}$ represents the prediction model trained for each application–node pair, mapping the selected features to the RTT estimate.

To build these models, we employ machine learning techniques such as Ridge regression, Random Forests and Support Vector Machines. Model selection minimizes RMSE while ensuring inference time remains bounded (i.e. below 1% of the mean task RTT), so that prediction overhead does not significantly affect task RTT. More complex models, including recurrent neural networks, are excluded because of their relatively high computational cost. There is one RTT predictor per application on each node.

C. Workload Predictors

The workload directed at an application, measured in tasks per second, plays a critical role in determining the resources required to maintain the quality of service. To enable proactive scaling in response to demand fluctuations, we integrate application-specific workload predictors, denoted as W_m for each application m. These predictors take as input the most recent k task arrival rates, $\{\lambda_{m,t-k}, \lambda_{m,t-k-1}, \ldots, \lambda_{m,t-1}\}$, and forecast the future l values $\{\lambda_{m,t}, \lambda_{m,t+1}, \ldots, \lambda_{m,t+l}\}$.

For that purpose, time-series forecasting techniques such as AutoRegressive Integrated Moving Average (ARIMA) [32] and Long Short-Term Memory (LSTM) networks [33] can be used to build these predictors. Each application is associated with a dedicated workload predictor, which is deployed on the master node (centralized placement) or any worker node (distributed placement).

D. Scheduling

Conventional scheduling strategies in container orchestration platforms, such as the default Kubernetes scheduler, primarily base application placement decisions on resource availability metrics such as CPU and memory usage. However, these approaches often neglect the impact of resource contention and hardware heterogeneity between nodes, which can significantly affect application performance. To address this limitation, performance-aware scheduling is introduced. This approach evaluates both the expected mean RTT and its variability when placing new applications.

Performance-awareness can be incorporated into the scheduler through an extension that leverages predictions of performance variability during the scoring phase of nodes. When a

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

new application instance m is considered for deployment on a node n, the current set of applications of the node $\mathbf{A}_n$ is extended to form $\mathbf{A}'_n = \mathbf{A}_n \cup \{m\}$. The scheduling decision then involves evaluating the predicted mean and standard deviation of the RTT variability for this updated configuration, denoted by $(\mu_{(n,m)}(\mathbf{A}'_n), \sigma_{(n,m)}(\mathbf{A}'_n))$. The scheduler selects the node $n \in \mathcal{N}$, where $\mathcal{N}$ is the set of available nodes, that minimizes the combined cost of predicted mean and variability, formally defined as:

$$\min_{n \in \mathcal{N},\ m \in \mathcal{M}} (\mu_{n,m}(\mathbf{A}'_n) + \sigma_{n,m}(\mathbf{A}'_n)) \tag{2}$$

This optimization is embedded in the scoring function of the scheduling process and ensures that the placement of the new application minimizes the mean RTT while accounting for interference effects with other co-located applications.

E. Load Balancing

Even when applications are initially scheduled on nodes that promise favorable performance, the actual RTT experienced by users can vary significantly due to the dynamic and intermittent nature of application workloads. Existing load balancing strategies, such as round-robin or random distribution, do not incorporate performance predictions, and hence may lead to suboptimal routing decisions under varying system conditions.

To overcome this limitation, performance-aware load balancing is proposed. In this approach, RTT predictors are used to estimate the expected RTT for each active application instance m at node n, denoted as $y_{(n,m)}$. These predictions are then used to dynamically assign routing weights that guide the distribution of incoming requests. Specifically, the weight $w_{(n,m)}$ assigned to each application instance is defined to be inversely proportional to its predicted RTT:

$$w_{(n,m)} = \frac{1}{y_{(n,m)}} \tag{3}$$

These weights are updated at runtime by a dedicated controller, allowing the load balancer (i.e. Istio load balancer) to prioritize instances with lower expected latency. This adaptive mechanism ensures that tasks are directed toward application instances most capable of meeting performance targets, thereby reducing overall RTT and improving system responsiveness. When the application processes only one request at a time (as in the SPA use case), the load balancer selects the instance with the lowest predicted RTT.

F. Autoscaler

Conventional autoscaling mechanisms (i.e. Kubernetes autoscaler) adjust the number of application replicas based on current resource utilization metrics, such as CPU or memory consumption. Although effective in certain scenarios, these reactive strategies inherently lag behind actual workload changes, as they respond only after performance has already been impacted. To overcome this limitation, predictive autoscaling integrates workload forecasting into the scaling logic, enabling proactive resource allocation.

Let $\lambda_{(m,t)}$ represent the predicted task arrival rate at time t for application m, and let R_m denote the maximum acceptable RTT for that application. To ensure that performance remains within this constraint, the number of replicas required at time t, denoted by $N_{(m,t)}$, is determined by:

$$N_{(m,t)} = \left\lceil \frac{\lambda_{(m,t)}}{\text{capacity}_m} \right\rceil, \tag{4}$$

where capacity_m is the maximum number of tasks that a single replica of the application m can process per unit of time while maintaining the RTT within the threshold R_m.

By forecasting $\lambda_{(m,t)}$ in advance, the autoscaler can adjust the replica count before performance degradation occurs. This proactive approach improves system responsiveness, prevents under-provisioning during workload spikes, and avoids unnecessary over-provisioning during periods of low demand.

VII. Experimental Set-Up

This section describes the applications, infrastructure, and workflow used in our experiments.

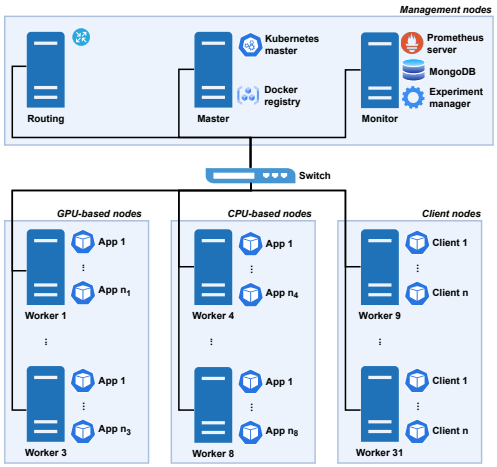

Fig. 3. Structure of the used infrastructure. The blue lines indicate the physical connectivity of the servers through a 1 Gbps Ethernet link.

We deploy five single-task SPA applications: Upload, MotionCor2, FFT Mock, gCTF, and ctffind4, each receiving POST requests with input data and returning responses upon completion. The image data, consisting of 98 2D cryo-electron microscopy (cryoEM) image stacks in MRC format, were acquired using a Glacios microscope with a Falcon4i detector. MRC [34] fraction files (897 MB) are processed by MotionCor2 and Upload, while integrated files (65 MB) are used by the other applications.

The Upload application transfers input files to worker nodes. MotionCor2 applies motion correction to image stacks, generating integrated frames. FFT Mock simulates lightweight real-time image processing by applying FFT-based shifts to MRC images. Both gCTF and ctffind4 estimate the Contrast Transfer Function (CTF) parameters, with gCTF leveraging GPU acceleration and ctffind4 relying solely on CPU execution.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SPECIFICATIONS OF INFRASTRUCTURE COMPONENTS.

Name	Model/Processor	Cores	RAM	GPU
Switch	Dell PowerConnect 5448	-	-	-
Routing	Intel Xeon X3430	3	4 GB	-
Master	Intel Xeon Silver 4109T	8	32 GB	-
Monitor	Intel Xeon Silver 4109T	8	78 GB	-
Worker-1	Intel Core i7-7700	4	16 GB	Tesla K20c
Worker-2	2xIntel Xeon E5-2637	4	24 GB	Tesla M40
Worker-3	Intel Core i9-14900K	32	128 GB	RTX4090
Worker-4	Intel Core i5-9600	4	12 GB	-
Worker-5	2xIntel Xeon E5504	8	24 GB	-
Worker-6	Intel Xeon Silver 4209T	8	24 GB	-
Worker-7	Intel Xeon Gold 6138T	16	64 GB	-
Worker-8	Intel Xeon Silver 4209T	8	24 GB	-
Worker-9-30	Intel Xeon X3430	4	4/8 GB	-

The infrastructure comprises 32 interconnected servers, including virtual and physical machines, as depicted in Fig. 3. The configuration of each server is shown in Table I. The Kubernetes cluster includes a master node and 30 worker nodes. Monitoring is handled by a dedicated virtual machine running Prometheus and MongoDB. Applications are deployed on Workers 1–8, with GPU-enabled workloads assigned to Workers 1–3, while Workers 9–30 host the client pods. Client pods are placed on separate nodes to emulate realistic latency and avoid interference, reflecting the typical real-world setup where clients run outside the resource cluster. Prometheus agents on each worker export monitoring data for analysis.

Our experiment workflow is orchestrated by an Experiment Manager pod. Applications are deployed across Workers 1–8 until node resources are saturated. Clients remain idle until activated by the manager, which controls execution stages and gradually increases workload. Each stage involves sending requests from clients to applications and measuring the RTT. The delay between tasks t_{wait} is randomly chosen within $[0, t_{max}]$, where t_{max} varies by application to induce resource contention. Randomizing t_{wait} avoids systematic bias that may arise from fixed request intervals [35]. RTT values are collected on the client side and stored in MongoDB; alternatively, Jaeger can measure in-cluster RTT, excluding external delays beyond cluster control. The manager periodically verifies whether sufficient samples are collected through CONFIRM [36] before moving onto the next workload stage. Once all stages are completed, the deployed pods are removed.

VIII. RESULTS

This section evaluates predictor accuracy and their impact on Kubernetes, with integration assessed through simulations rather than live deployment.

A. Variability Predictions

The first subplot in Figure 4 shows the RMSE of the variability predictions using three machine learning models: ridge regression (referred to as Ridge), RF, and SVM. Each model estimates both the mean and standard deviation of the RTT for the SPA applications under different co-location scenarios on each worker node. Each box plot illustrates the RMSE of each variability predictor across nodes, with observed values ranging from 5% to 22%. This range is due to differences in dataset sizes and the complexity of co-located scenarios on each node. Among the models, RF achieves the lowest error, with an average RMSE of 10% in both predicted metrics (mean and standard deviation), while having an RMSE range similar to that of the other models. The predictors are fast, achieving an average inference time of 82 μs, which makes them suitable for real-time scheduling. These results highlight the practicality of integrating lightweight node-level variability predictors into orchestration frameworks to improve scheduling decisions under dynamic allocation conditions.

B. RTT Predictions

The second subplot in Figure 4 presents the RMSE of the RTT predictions using the same three models. Each box plot summarizes the prediction error of the predictors for each application and node. Once again, RF achieves the lowest average RMSE across all RTT predictors, with a mean RMSE of 12.5%, outperforming both Ridge Regression and SVM. Although Ridge Regression shows a slightly lower mean RMSE in some cases, it exhibits a wider RMSE range across different predictors, indicating less consistent performance. The inference time for the RTT predictors is just 36μ s, allowing for fast decision making per task. This low-latency prediction capability supports fast application instance selection, allowing the orchestrator to quickly adapt to changing runtime conditions while maintaining high prediction accuracy.

C. Simulating Performance-Aware Scheduling

To evaluate performance-aware scheduling, we developed a simulation framework that models a heterogeneous cluster where nodes differ in CPU, memory, and processing speed. This framework quantifies the potential benefits of integrating variability predictors into the Kubernetes scheduler. Applications are represented as pods with specific resource demands and base RTTs derived from empirical SPA measurements. Co-location effects are modeled through an interference matrix obtained from testbed experiments, which captures how the RTT mean and variability increase when applications share a node.

The third subplot of Figure 4 shows how the overall task completion time evolves with increasing numbers of applications (up to 1000), comparing the default Kubernetes scheduler against the performance-aware scheduler. As the workload scales, the performance gap between the two approaches widens significantly. The completion time for applications scheduled by the performance-aware scheduler increases at a significantly slower rate than those scheduled by the default Kubernetes scheduler. The simulation indicates that the use of performance predictors reduces the overall completion times by 50% to 78%. This improvement is attributed to the ability of the performance-aware scheduler to estimate the RTT mean and variability in advance, leading to more efficient application placement while maintaining the same resource usage as the default scheduler. Additionally, the reduction in total completion times achieved by the performance-aware

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

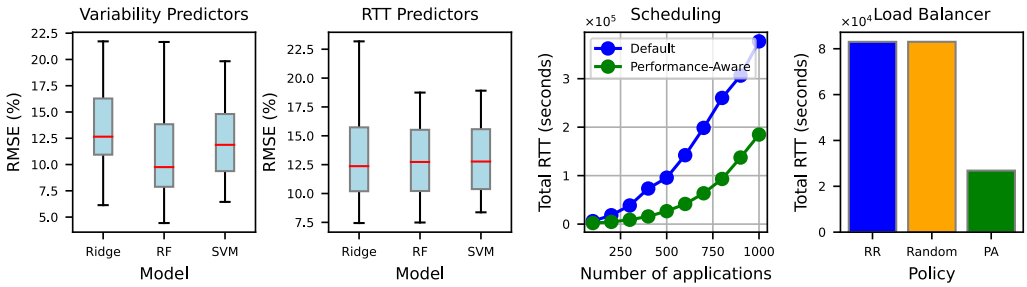

Fig. 4. The four subplots show, from left to right: The first subplot shows the RMSE for the variability predictors. The second subplot shows the RMSE for the RTT predictors. The third subplot compares the default Kubernetes and performance-aware scheduler. The fourth subplot compares load balancers with round-robin (RR), random and performance-aware (PA) policy.

scheduler shows that more applications can be executed within the same time frame compared to the default scheduler.

D. Simulating Performance-Aware Load Balancing

A similar framework is used to assess performance-aware load balancing, reusing the same heterogeneous cluster model, application profiles, and RTT computation method as in the scheduling simulations. In this case, multiple replicas of each application are randomly placed across nodes at the start of each trial to isolate the impact of the balancing strategy itself. The fourth subplot of Figure 4 compares the total RTT of tasks under three different load balancing policies: round-robin (RR), random, and performance-aware (PA), with an accuracy of 80%.

The results in the fourth subplot of Figure 4 indicate that, while the total RTT for tasks is similar under both the round-robin and random policies, it is significantly lower under the performance-aware policy. Specifically, the simulation demonstrates a reduction 68% in overall RTT when performance predictors are used. This improvement is due to the ability of the performance-aware policy to account for interference from co-located applications and the capabilities of each node (e.g., CPU speed), enabling the selection of the most suitable application instance at the time of task submission. Consequently, resource utilization and energy efficiency are improved by reducing unnecessary processing and reducing task completion times. This approach also improves system throughput, enabling more tasks to be processed within the same time frame compared to round-robin and random scheduling policies.

IX. Conclusions

In this paper, we presented an integrated architecture that enhances scheduling and load balancing with performance awareness. We developed two types of performance predictors to estimate both the overall RTT variability and the RTT of individual tasks. Our study specifically focused on an electron microscopy use case, where applications operate on subsecond timescales. Our proof-of-concept suggests that our performance predictors are accurate and fast, leading to improved near-real-time scheduling and load-balancing decisions. The effectiveness of our approach was validated through simulation, showing significant improvements in resource management and application performance. In future work, we will integrate the predictors into Kubernetes components and evaluate their performance on real clusters with heterogeneous nodes and diverse applications. We also plan to measure their overhead, assess interference with running workloads, and compare them with more advanced decision-making policies.

Acknowledgments

This publication is part of the project ADAPTOR: Autonomous Distribution Architecture on Progressing Topologies and Optimization of Resources (with project number 18651 of the research Open Technology Programme which is (partly) financed by the Dutch Research Council (NWO). The image data used for the experiments was acquired at Thermo Fisher Scientific (Eindhoven, The Netherlands). The computational experiments were executed at the Eindhoven University of Technology (Eindhoven, The Netherlands).

References

[1] D. Balouek-Thomert *et al.*, "Towards a computing continuum: Enabling edge-to-cloud integration for data-driven workflows," *The International Journal of High Performance Computing Applications*, vol. 33, no. 6, pp. 1159–1174, Nov. 1, 2019.

[2] J. Dean *et al.*, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Oct. 1, 2013.

[3] A. Kuriata *et al.*, "Predictable performance for QoS-sensitive, scalable, multi-tenant function-as-a-service deployments," in *Agile Processes in Software Engineering and Extreme Programming – Workshops*, M. Paasivaara *et al.*, Eds., vol. 396, Series Title: Lecture Notes in Business Information Processing, Cham: Springer International Publishing, 2020, pp. 133–140.

[4] S. Chunduri *et al.*, "Run-to-run variability on xeon phi based cray XC systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Denver Colorado: ACM, Nov. 12, 2017, pp. 1–13.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.

[5] H. Mohamed *et al.*, *End-to-End Latency Prediction of Microservices Workflow on Kubernetes: A Comparative Evaluation of Machine Learning Models and Resource Metrics*. Jan. 5, 2021.

[6] Y. Fu *et al.*, "Progress-based container scheduling for short-lived applications in a kubernetes cluster," in *2019 IEEE International Conference on Big Data (Big Data)*, Los Angeles, CA, USA: IEEE, Dec. 2019, pp. 278–287.

[7] Y. Wang *et al.*, "Predicting new workload or CPU performance by analyzing public datasets," *ACM Transactions on Architecture and Code Optimization*, vol. 15, no. 4, pp. 1–21, Dec. 31, 2018.

[8] S. Kianpisheh *et al.*, "Resource availability prediction in distributed systems: An approach for modeling non-stationary transition probabilities," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 8, pp. 2357–2372, Dec. 2017.

[9] W. Zheng *et al.*, "End-to-end delay modeling via leveraging competitive interaction among network flows," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 1634–1647, Apr. 2024.

[10] C. Delimitrou *et al.*, "Quasar: Resource-efficient and QoS-aware cluster management," in *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*, ser. ASPLOS '14, New York, NY, USA: Association for Computing Machinery, Oct. 24, 2014, pp. 127–144.

[11] D. Dauwe *et al.*, "HPC node performance and energy modeling with the co-location of applications," *The Journal of Supercomputing*, vol. 72, no. 12, pp. 4771–4809, Dec. 1, 2016.

[12] P. Townend *et al.*, "Invited paper: Improving data center efficiency through holistic scheduling in kubernetes," in *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, San Francisco East Bay, CA, USA: IEEE, Apr. 2019, pp. 156–15 610.

[13] I. Harichane *et al.*, "A proposal of kubernetes scheduler using machine-learning on CPU/GPU cluster," in *Intelligent Algorithms in Software Engineering*, R. Silhavy, Ed., Cham: Springer International Publishing, 2020, pp. 567–580.

[14] B. Alankar *et al.*, "Experimental setup for investigating the efficient load balancing algorithms on virtual cloud," *Sensors*, vol. 20, no. 24, p. 7342, Jan. 2020.

[15] L. Laukka *et al.*, "Load balancing traffic among kubernetes replicas by utilizing workload estimation," in *2023 IEEE Conference on Standards for Communications and Networking (CSCN)*, ISSN: 2644-3252, Aug. 2023, pp. 353–356.

[16] Z. Yao *et al.*, "HLB: Toward load-aware load balancing," *IEEE/ACM Transactions on Networking*, vol. 30, no. 6, pp. 2658–2673, Sep. 2022.

[17] J. Chen *et al.*, "A context-aware approach to task scheduling for time series data prediction in mobile edge computing," *Mobile Networks and Applications*, vol. 28, no. 1, pp. 421–431, Feb. 1, 2023.

[18] J. Gao *et al.*, "Machine learning based workload prediction in cloud computing," in *2020 29th International Conference on Computer Communications and Networks (ICCCN)*, ISSN: 2637-9430, Dec. 2020, pp. 1–9.

[19] M. Xu *et al.*, "esDNN: Deep neural network based multivariate workload prediction in cloud computing environments," *ACM Trans. Internet Technol.*, vol. 22, no. 3, 75:1–75:24, May 17, 2022.

[20] P. Yazdanian *et al.*, "E2lg: A multiscale ensemble of LSTM/GAN deep learning architecture for multistep-ahead cloud workload prediction," *The Journal of Supercomputing*, vol. 77, no. 10, pp. 11 052–11 082, Oct. 1, 2021.

[21] L. Toka *et al.*, "Adaptive AI-based auto-scaling for Kubernetes," en, in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, Melbourne, Australia: IEEE, May 2020, pp. 599–608.

[22] D. G. Reichelt *et al.*, "Automated identification of performance changes at code level," in *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, ISSN: 2693-9177, Sep. 2022, pp. 916–925.

[23] L. Meng *et al.*, "Detecting anomalies in microservices with execution trace comparison," *Future Generation Computer Systems*, vol. 116, pp. 291–301, Mar. 1, 2021.

[24] *Kubernetes Project*, https://kubernetes.io.

[25] *Prometheus*, https://prometheus.io/.

[26] *jaegertracing*, https://www.jaegertracing.io/.

[27] *Istio*, https://istio.io/.

[28] P. Giannakopoulos *et al.*, "perfCorrelate: Performance variability correlation framework," *Future Generation Computer Systems*, vol. 170, p. 107 827, Sep. 1, 2025.

[29] A. E. Hoerl *et al.*, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 42, no. 1, pp. 80–86, 2000.

[30] C. Cortes *et al.*, "Support-vector networks," *Machine learning*, vol. 20, no. 3, pp. 273–297, 1995.

[31] T. K. Ho, "Random decision forests," in *Proceedings of 3rd international conference on document analysis and recognition*, IEEE, vol. 1, 1995, pp. 278–282.

[32] G. M. Jenkins, "Autoregressive–integrated moving average (arima) models," in *Wiley StatsRef: Statistics Reference Online*. John Wiley & Sons, Ltd, 2014.

[33] S. Hochreiter *et al.*, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[34] A. Cheng *et al.*, "MRC2014: Extensions to the MRC format header for electron cryo-microscopy and tomography," *J Struct Biol*, vol. 192, no. 2, pp. 146–150, Nov. 2015.

[35] T. Mytkowicz *et al.*, "Producing wrong data without doing anything obviously wrong!" *SIGPLAN Not.*, vol. 44, no. 3, pp. 265–276, Mar. 2009.

[36] A. Maricq *et al.*, "Taming performance variability," pp. 409–425, Oct. 2018.

Authorized licensed use limited to: MIT-World Peace University. Downloaded on February 18,2026 at 06:08:24 UTC from IEEE Xplore. Restrictions apply.